Journal of Systems Architecture 82 (2018) 37–48

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Hypervisor for consolidating real-time automotive control units: Its procedure, implications and hidden pitfalls

Arun Kumar Sundar Rajan[a,e,*], Armin Feucht[b], Lothar Gamer[c], Idriz Smaili[d], Nirmala Devi M.[e]

[a] Robert Bosch Engineering and Business Solutions Private Limited, Coimbatore, Tamil Nadu 641035, India
[b] Bosch Engineering GmbH, Postfach 13 50, Heilbronn 74003, Germany
[c] Robert Bosch GmbH, Robert-Bosch-Strasse 2, Schwieberdingen, Baden-Wuerttemberg 71701, Germany
[d] Robert Bosch AG, Göllnergasse 15-17, Vienna 1030, Austria
[e] Department of Electronics and Communications Engineering, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Coimbatore, India

ABSTRACT

The future of automotive industry has a major focus on computing innovations such as autonomous driving, connectivity and mobility. These complex innovations integrate hardware and software components of different automotive safety integrity levels. Although standards like AUTOSAR provides flexibility in integrating different software components, it compromises on isolation from software interference. Virtualization is a technique, used for several decades in server domains, ensuring integration of heterogeneous subsystems without interference. Until recent years, inheriting virtualization onto real-time systems has been avoided due to its overhead and impact in-terms of timing behavior. With the usage of multi- and many-cores in the automotive industry, virtualization is gaining more importance to handle the upcoming innovations with incredible increased complexity and the needed computation power. This paper describes the technique for bringing virtualization into an automotive state-of-the-art multicore controller, which does not support virtualization. We have used two automotive state-of-the-art heterogeneous applications: engine electronic control unit and vehicle control unit, and a prototype version of hypervisor from ETAS-GmbH. Important aspects in virtualization like system-startup, handling of interrupts and traps, facilitating input–output access are discussed. Finally, the performance of a virtualized system in terms of core loading, interrupt and task timing parameters are evaluated and compared.

1. Introduction

With more and more electrical and electronic (E/E) innovations happening in the automotive industry, the software complexity and the number of components, e.g. electronic sensors, control units and actuators are increasing exponentially. For safety and reliability purposes, software subsystems are distributed across multiple control units which make the vehicle control system more modularized. However, the future of the automotive industry is towards a centralized vehicle E/E architecture [1]–which will save development cost as well as bring in faster communication and computation power [2]. Hence it is necessary to unite similar functional units of different automotive safety integrity level (ASIL) categories [3] into domains, which are then controlled by a centralized unit [4].

Until recent years, each ECU (electronic control unit) provider followed their own non-standard development procedure, which makes integration of subsystems more complex. This led to the emergence of AUTOSAR [5], which creates and establishes an open and standardized software architecture for automotive ECUs, whereby an original equipment manufacturer (OEM) will have a freedom (plug and use) to integrate software components. Although AUTOSAR provides flexibility in building and integrating different software components, it compromises on complete isolation from software interference [1].

Virtualization is a technique that has been used for several decades in servers and other computer application domains, which ensures integration of heterogeneous software systems with isolation from interference [6]. Inheriting the virtualization concept onto real-time embedded systems has been restricted due to overheads on timing performance and resource consumption [7,8]. With usage of faster multicore and many-core chips in the automotive industry, virtualization seems to be a possible endeavor [9]. Virtualization is achieved through an introduction of a software layer termed as *Hypervisor* or *Virtual Machine Monitor* into the system, whereby hypervisor interfaces between hardware and the applications, governing the spatial and

* Corresponding author at: Department of Electronics and Communications Engineering, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Coimbatore, Tamil Nadu 641112, India
E-mail address: s_arunkumar@cb.amrita.edu (A.K. Sundar Rajan).

https://doi.org/10.1016/j.sysarc.2018.01.001
Received 2 June 2017; Received in revised form 5 November 2017; Accepted 8 January 2018
Available online 09 January 2018
1383-7621/ © 2018 Elsevier B.V. All rights reserved.

temporal separation and managing the resource utilization. With hypervisor, each application acts independently, virtualizing the control over the hardware.

An ideal virtualized system has multiple advantages like, plug and use of applications without affecting other applications, timely share of resources among applications, freedom from interference of malicious applications and many more [10]. However, when virtualizing a hard real-time embedded system with huge legacy software, the behavior is quite different. Aim of our work is to identify the possible failures and real-time issues that arise with virtualizing automotive state-of-the-art applications over ASIL-D controllers. For the evaluation, two automotive state-of-the-art heterogeneous applications namely, engine ECU (E-ECU) and vehicle control unit (VCU) and a prototype version of hypervisor from ETAS GmbH [11] are used. The controller is from Infineon AURIX family TC29x [12] which is ASIL-D compliant.

Important aspects in virtualization like system startup behavior, handling of hardware interrupts and traps, facilitating input and output access to each application are elaborated in this paper. We also discuss about the limitations of the existing automotive state-of-the-art controllers towards virtualization. Finally, performance of the virtualized system in terms of core loading, interrupt and task timing parameters are evaluated and compared.

This paper is organized as follows: Section 2 outlines about the real-time embedded hypervisors and explains in context with ETAS hypervisor and its properties. Section 3 describes about the limitation with ASIL-D automotive standard microcontrollers and techniques for virtualization in such hardware. Section 4 provides the implementation details for setting up a virtualized system. The performance evaluations are discussed in Section 5 and summarized in Section 6.

2. The hypervisor–an overview

Since 1960, hypervisor has been exploited in main-frame and server application domains; also recently in consumer electronics [6]. In these applications, virtualization has offered several advantages such as the ability to run several operating systems in coexistence on a single hardware and at the same time isolating them from each other in terms of security, safety and fault containment. However, the usage of hypervisor for hard real-time embedded systems has been always limited by constraints like memory overhead and performance lag [7]. With advancement in microcontroller technology, such as multicore and many-core embedded controllers running at few hundreds of megahertz, embedded developers have started exploiting hypervisors in hard real-time embedded applications [13].

There are four possible ways to classify a hypervisor [14,15], namely:

Type 1 or bare metal hypervisor: The hypervisor layer encapsulates the hardware. The operating system (OS) is specific to the application, which is called as *guest OS*, as shown in Fig. 1.a. The hypervisor layer serves the guest application requests for access to the hardware and also forwards the hardware interrupts to the specified guest OS.

Type 2 or hosted hypervisor: Hardware comes with an OS called *host OS* and the hypervisor is implemented on top of the host OS. Hypervisor cannot directly access the hardware and it has to use the services provided by the host OS, as shown in Fig. 1.b. The mounted applications can have their own OS (*guest OS*).

Full virtualization: The hypervisor supports migration without any changes to the application. In order to achieve full virtualization, hardware must possess virtualization extensions.

E.g.: Virtualization extensions like ARM-VE, Intel VT-x and VT-d.

Para virtualization: The hypervisor requires modification in the application by incorporating hypervisor calls.

E.g.: If an application wants to access the hardware resources, its request has to go via hypervisor layer. For this, the application code accessing directly the I/O devices has to be replaced with hypervisor API and then the communication happens between application and hardware via hypervisor layer.

There are several hypervisors for embedded systems, some of them are commercialized and many are still of open source platforms. We have come across some open source hypervisors such as Xen [16], XtratuM [17,18], OKL4 [19], Linux KVM [20], Proteus Hypervisor [21], Xvisor [14] which are experimented on different chips like Leon processor, ARM based processor etc. and supporting full/para virtualization. There are also commercial hypervisors like Green Hills' Integrity [22], Freescale's Topaz [23], and Wind River Hypervisor [24]. From hardware perspective, Intel and AMD [25] have been supplying processors for server applications, supporting virtualization. ARMv7 and its succeeding architecture versions support virtualization by providing additional controller modes [26,27].

ETAS GmbH has provided an automotive state-of-the-art hypervisor (RTA-HVR) prototype library, which of Type 1 category and supports paravirtualization [11]. Comparison of some renowned embedded hypervisors is presented in Table 1.

RTA-HVR is specifically tailored to suit hard real-time automotive requirements and for hardware without virtualization support. This means RTA-HVR can run on microcontrollers with a simple memory protection unit (MPU) and not a full memory management unit (MMU). Further, it is AUTOSAR [5] compliant and inputs to this hypervisor are configurable, in terms of interrupts, traps, memory protection regions and input-output peripheral regions. Based on the configuration, hypervisor functions are generated out of the library. Interactions between the hardware and applications happen via hypervisor's application programming interface (API), whose definition can be adapted according to the use-cases. Additional user specific functions can also be integrated into this hypervisor.

Porting of ETAS hypervisor on ASIL-D Infineon AURIX family TC27x with two automotive state-of-the-art applications, namely powertrain and BMW AUTOSAR CORE (BAC4), was earlier performed by Reinhard and Morgan [11]. Here, a non-intrusive mechanism of porting the

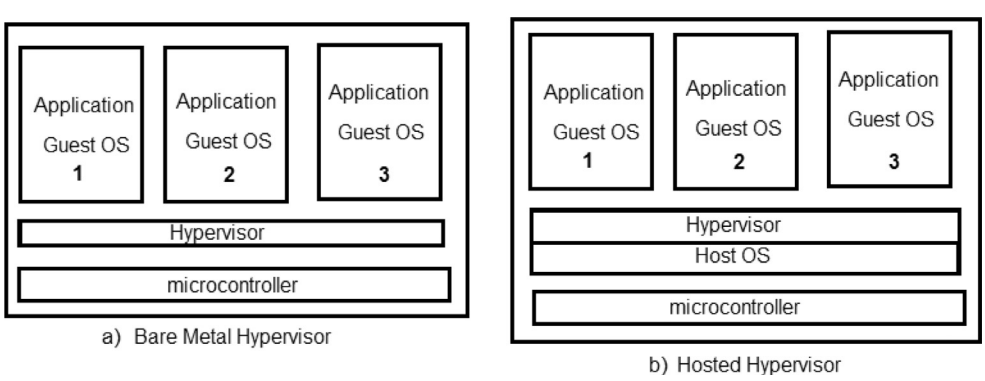

Fig. 1. Block diagram on type1 and type2 hypervisor.

Table 1
Comparison with renowned embedded hypervisors.

	RTA-HVR [11]	Integrity multivisor [22]	Wind river hypervisor [24]	XEN hypervisor [28]
Type	BareMetal	BareMetal	BareMetal	BareMetal
Version	Prototype version	Commercial	Commercial	Open source
Supported Architecture	Aurix	Intel, Power and ARM	Intel, Power and ARM	Application specific
Virtualization Type	Paravirtualization with minimal migration cost.	Paravirtualization (PV) and hardware-assisted full virtualization (HVM).	Paravirtualization (PV) and hardware-assisted full virtualization (HVM).	Paravirtualization (PV) with huge migration cost.
Hardware accessibility	Supports direct and paravirtualized hardware access. Direct access is possible using the memory protection unit of the controller.	Requires hardware assisted extensions for full virtualization.	Requires hardware assisted extensions for full virtualization	Applications cannot directly interact with the hardware. Only Domain0 contains all drivers for hardware access and applications have to communicate with Domain0 for hardware access.
	For paravirtualized access, emulation of the instruction saves migration cost but reduces the response time.	For processors lacking virtualization mode assistance, Multivisor requires intrusive modifications to the application and recompilation.	Also supports paravirtualization, where the application is modified and recompiled	The Xen Project Hypervisor is not usable without Domain0 and requires application recompilation
Use cases	Developed to satisfy ISO26262 - Automotive standard and AUTOSAR.	Multivisor is used for in-vehicle infotainment and is not yet ISO26262 complaint	Offers virtualization and ECU consolidation solutions for customers who need to comply with automotive safety and security constraints.	Open source and certification is based on the end product.
	Definition for the hypervisor API's that interacts with hardware and applications are open and can be developed according to the application.			

applications exploiting the MPU of the hardware is proposed. This technique depends on the span of the application that is to be virtualized. Bigger applications are spread across multiple memory regions and also need access to many peripherals; which will exceed the MPU's capacity for mapping address (memory and peripheral) ranges into the applications address space.

To overcome this limitation, we have developed a mechanism termed as *IO emulation* where the input/output access to peripheral address region is performed by emulating the trap handler. It is a non-intrusive technique and supports easy migration of the application. In subsequent sections, detailed information about the limitations in MPU's capacity, principle behind IO emulation and experimental results are provided. It is important to know the amount of overhead a virtualization can impinge to the system between MPU based and IO emulation based access and the same is also discussed.

3. Significant design aspects

There are many ECUs deployed in the car for a dedicated purpose, each executing in their own hardware. For example, the E-ECU is responsible for powertrain functionalities and the VCU takes care about the vehicle dynamics. From the hypervisor perspective, each subsystem like E-ECU or VCU will be treated as an individual application. Both these automotive applications have been evolved over the years independently, based on different customer needs, methods and tools. The new challenge is to integrate them on to a single hardware with minimal cost. In addition, the consolidation must guarantee parallel software execution with spatial (memory regions) and temporal (time) isolation between the applications; which has led hypervisor into automotive software domain. Hypervisor can encapsulate each application and provide a feel to each application of owning the complete hardware dedicatedly.

3.1. Why TC29x controller?

As per ISO26262–functional safety for automotive standards, an automotive grade control system must function correctly to avoid hazardous situations and must demonstrate an ability to detect and manage faults. Accordingly, there are four levels of safety integrity levels starting from ASIL A to ASIL D, with ASIL D representing the highest integrity requirements with product's single points of failure in the entire system is less than 1%. In powertrain and vehicle control applications, where highly safety critical functions are required in the actuation area, ASIL D [29] is demanded.

For ASIL D, processors handling safety-critical functions need to operate in a lockstep mode. Lockstep mode requires a second “shadow” core, which computes and compares the output to detect if a fault has occurred. This lockstep mode must also account for potential failures caused by events that could affect both cores at the same time. (E.g. byzantine faults).

Intel and AMD have modern processor architectures that possess hardware assisted extensions for virtualization. ARMv7 and its succeeding architecture versions support virtualization for embedded applications, by providing additional controller mode ***hyp*** with higher privileges [26,27]. However, these processors do not satisfy the ASIL-D [29] requirement which is mandatory for automotive safety-critical systems, e.g. E-ECU.

There are only few micro controller unit (MCU) solutions that are of ASIL-D category, namely

- Freescales' MPC5643L 32-bit MCU based on Power architecture, [30]
- STMicroelectronics' SPC5 MCU based on Power architecture, [31]
- Texas Instruments' Hercules TMS570 based on ARM Cortex-R4 architecture, [32]
- Infineon's Aurix based on 32-bit TriCore architecture [33] and

- Renesas' RH850/P1x series of 32-bit MCUs. [34]

These MCUs do not have the hardware-assisted extensions for virtualization. Hence, paravirtualization is the possible solution and steps needed for paravirtualizing the system is explained in Section 4.

Note: ARM v8-R architecture is a real-time embedded processor core designed with a hardware-assisted virtualization mode. This architecture is incorporated in Cortex-R52 processor [35], designed to address ASIL-D, which features dual-core lockstep with comparators to monitor the processors and to detect faults. ARM is certainly seeking to expand its automotive footprint and is expected to meet volume production requirements of Cortex-R52 by 2020 [36].

3.2. Exploiting hardware resources for virtualization

For processors lacking hypervisor mode assistance, a different strategy has to be exploited. Our demonstration is based on an automotive state-of-the-art multicore controller from Infineon AURIX family TC29x [12], with three independent 32-bit TriCore™ CPUs (i.e., execution units–cores), designed to meet ASIL D standard [3] and increase the performance at the same time. Block diagram of the AURIX TC29x controller is shown in Fig. 2.

Each core has its own address and data register sets, local memory region for code and data, shared memory regions between the cores, checker or lockstep core and many more features. However, this controller does not possess the instruction set architecture (ISA) [37] to support virtualization and hence it becomes a challenge to implement virtualization. The basic controller's support required for virtualization is to facilitate access-control over the memory and peripherals regions. It can be either provided by additional controller modes or memory access-control peripherals. AURIX TC29x controller has a memory protection unit (MPU) and three controller operating modes namely *user-0, user-1* and *supervisor* mode [12]:

User-0 Mode: This mode does not allow access to any peripheral devices. Enabling or disabling of interrupts is also not permitted.
User-1 Mode: This mode allows access to unprotected peripherals like read/write access to serial port, read access to timer and I/O status registers. It also allows disabling interrupts for a shorter period, which can be overridden by system control register.
Supervisor Mode: This mode permits read/write access to system registers and all peripheral devices. It also allows enabling and disabling of interrupts.

From the hypervisor perspective, E-ECU and VCU are just two applications running independently and it is the role of hypervisor to facilitate the hardware accesses requested by the applications. Further, applications should not communicate between each other without the knowledge of hypervisor. Hence, the applications are made to execute in a lesser privileged *user-1* mode and hypervisor runs in *supervisor* mode.

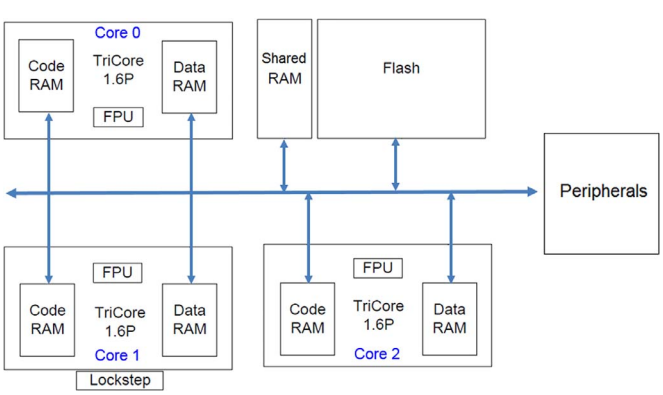

Fig. 2. Simplified block diagram of AURIX TC29x controller.

3.3. Limitation over the MPU register-pairs

MPU protects the access of an application (read, write and execute) to code and data regions. MPU understands the memory regions to be protected by using the base/limit register pairs, which holds the access type and memory address range details. Each MPU base/limit register pair has to be configured statically. Violation of access to the protected memory region will end up in a trap.

For a non-intrusive implementation of a virtualized system, the entire address range in terms of flash, RAM and peripheral address space has to be protected using MPU. Each core in AURIX TC29x has four MPU protection register sets (PRS), which can be interchanged only in *supervisor* mode by writing to processor's state register.

Each PRS consists of:

- 8 Code Protection Ranges (CPR), each with related XE (execute enable) access flag,
- 16 Data Protection Ranges (DPR), each with related RE (read enable) and WE (write enable) access flags.

Protection register sets are allocated such that,

- PRS 0 is the hypervisor, running in *supervisor* mode.
- PRS 1 is the trusted partition code, running in *user-1* mode.
- PRS 2 is the untrusted partition code, running in *user-1* mode.
- PRS 3 is unused.

OS makes the differentiation between trusted partition code and untrusted partition code; controlling the access using DPR and CPR access flag configuration. *Note: Configuration of each PRS along with its access and memory address ranges is static and does not change during runtime.*

For instruction fetches, the program counter (PC) value for the fetch is checked against the selected code protection ranges (CPR) of the current protection set. For load and store operations, data address values are checked against the selected data protection ranges (DPR) of the current protection set. When the address is found to fall outside of all of the selected ranges, then permission for the access is denied. When an address is found to fall within one of the selected ranges, the associated access permission is checked and the access is allowed or denied as appropriate [12].

The real limitation with MPU implementation is the number of base/limit register pairs available to configure the address ranges for an application. Typical allocation of base/limit register pairs per core to the minimal memory address range requirement of an application in TC29x [12] is presented in Table 2. It is observed that only 2 DPRs are remaining free, which is not sufficient to partition application specific peripheral address regions. Peripherals are distributed in address range starting from 0xF000 0000 to 0xFFFF FFFF. [12]

To overcome this limitation, we have exploited a mechanism termed as *IO emulation* where the input/output access to peripheral address region is performed by emulating the trap handler. It is a non-intrusive technique and supports easy migration of the application. It is also not limited by the number of address range specifiers. Details about the IO emulation are explained in *Section 4.5.3*.

4. Implementation

In this section, various technical aspects that are needed for incorporating hypervisor based solutions are presented. It begins with an explanation about the application-to-core allocation strategy followed by the partitioning of code and data regions between the applications using the MPU. Later, modifications needed at the boot-up sequence are

Table 2
Allocation of MPU's base/limit pair registers.

Code protection regions–XE access		Data protection regions–RE/WE access			
CPR0	Reserved	DPR0	Reserved	DPR8	RAM1_PRAM1
CPR1	PRAM_ALL	**DPR1**	**Free**	DPR9	RAM0_PRAM0
CPR2	RAM2_PRAM2	**DPR2**	**Free**	DPR10	System RAM
CPR3	RAM1_PRAM1	DPR3	EDRAM	DPR11	EEPROM
CPR4	RAM0_PRAM0	DPR4	Data logger	DPR12	DataFlash
CPR5	CodeFlash–cached access	DPR5	PRAM_ALL	DPR13	CodeFlash–noncached access
CPR6	CodeFlash–noncached access	DPR6	RAM_ALL	DPR14	CodeFlash–cached access
CPR7	Reserved	DPR7	RAM2_PRAM2	DPR15	reserved

PRAM–program RAM; RAMx_PRAMx–shared region to be used for RAM and program RAM.
EDRAM–nonvolatile RAM; PRAM/RAM_ALL–dedicated usage to be used for RAM or program RAM.

described. Further, four different techniques for facilitating the input-output access are discussed and compared. The interrupt is an important hardware mechanism to notify the software about an event. In our virtualized system, which consists of a hypervisor and two real-time subsystems (E-ECU and VCU), it is crucial to specify who is handling the interrupts and the procedure is also explained here. Another critical player of embedded system is a trap, which can be triggered by hardware as well as by software and emulating their handlers for specific purposes is addressed at the end of this section.

4.1. Core allocation

Infineon AURIX family TC29x multicore controller has three independent 32-bit TriCore™ CPUs, as formerly presented in Fig. 2. Compared to VCU, in terms of code density and real-time usage of cores, E-ECU needs more computational cores and must satisfy ASIL D requirement. Hence core 0 and core 1 (with lockstep core) are allocated to E-ECU, while core 2 for VCU functions. The new software distribution is depicted in Fig. 3. Hypervisor has its instance in all three cores, saving the clock ticks by accessing its own local memory.

In this demonstrator, each core is partitioned as a virtual machine (VM) and subsequently each VM is configured separately. There are five configurations per VM, namely

a) Interrupt handlers and their priority,
b) Trap handlers,
c) MPU controlled memory regions,
d) Regions under input-output (IO) configuration,
e) Starting address location for hypervisor main function.

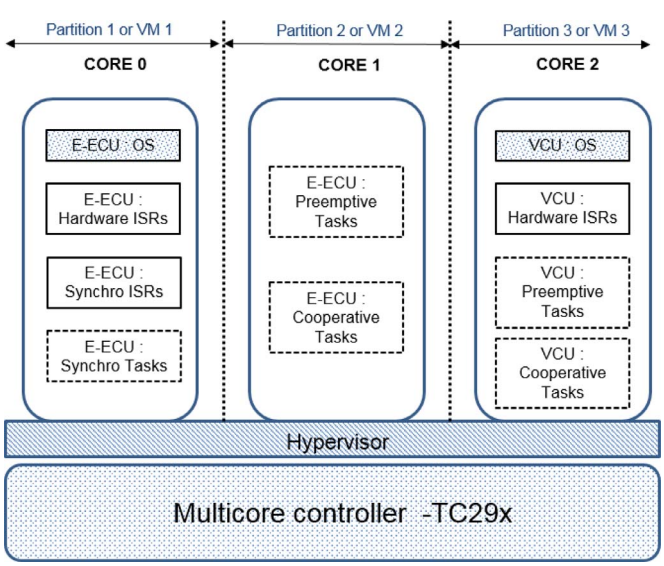

Fig. 3. Software distribution among E-ECU and VCU.

Since E-ECU software is spread across core 0 and core 1, hypervisor configurations for core 0 and core 1 will be alike. *Note: It is possible in some hypervisors to further partition a core into multiple VMs; however, it is not possible in this prototype version of hypervisor.*

4.2. Securing the partitioned memory regions

Memory regions such as flash and RAM have to be protected between the applications from spatial interference. As discussed in *Section 3.3*, MPU comes in handy and in case of any access violation, a trap will be generated by the hardware. RTA-HVR supports the development of memory configuration files in AUTOSAR (*.arxml) format [5], with following options for MPU:

a) Address range for base/limit register pairs, and
b) Read, write and execute access permissions for the configured address range.

An example of configuration tags used in MPU base/limit register pairs are,

- MpuRegionStart – Address of the start of the MPU region.
- MpuRegionEnd – Address of the end (i.e. the last legal location) of the MPU region.
- MpuRegionRead – Stating whether or not the region is readable.
- MpuRegionWrite – Stating whether or not the region is writeable.
- MpuRegionExecute – Stating whether or not the region is executable.

4.3. Startup sequence

Even in a multicore system, at start-up (power on or reset) only one core will begin its execution. After all the hardware checks and initialization is complete, comes the start-up for other cores, as depicted in Fig. 4. However in a virtualized system, hypervisor should know the number of cores that it will handle and the hypervisor plays the role of starting the cores. Hence, the start-up sequence with inclusion of hypervisor has to be modified and the same is depicted in Fig. 5.

In this virtualized system, VCU is having a headless start without any hardware based boot codes. Hardware is checked only by the boot block present at core 0. Secondly, the hypervisor starts the cores at *supervisor* mode and then after the hardware initialization, hypervisor switches the execution to *user-1* mode. It is essential to know the dependency on instrumenting tools such as runtime measurement tools, which needs initialization in *supervisor* mode and to bring in their initialization before hypervisor switches the core to *user-1* mode.

4.4. Handling of traps and interrupts

Peripherals communicate with the cores using hardware interrupts, which are defined in TC29x by service request interrupt source (SRC)

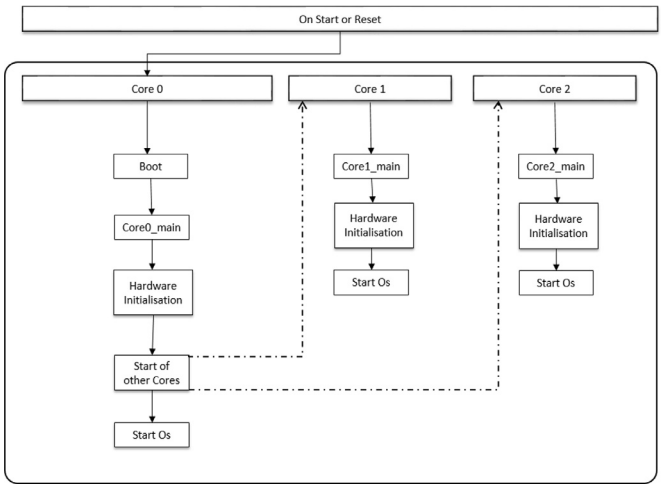

Fig. 4. Conventional multicore start-up sequence.

registers. There is one SRC register per interrupt source with configurable priority. Based on the priority, entries for interrupt service routine (ISR) are updated in the vector table. In multicore environment, each core has its own interrupt vector table and trap vector table [38]. The interrupt is routed to the corresponding core based on the target core configuration at SRC register.

In a conventional procedure, applications directly respond to the interrupt via its ISRs. However, in the virtualized system, interrupt and/or trap is first handled at the hypervisor layer and not by the

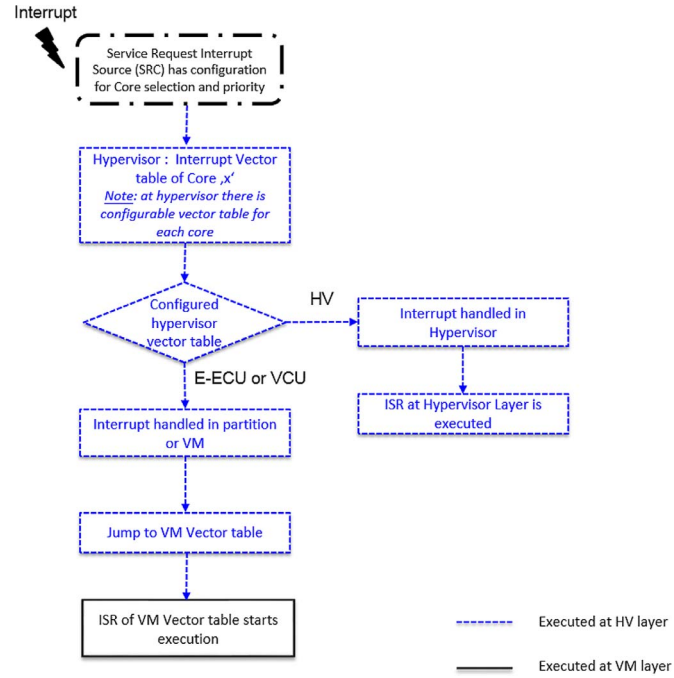

Fig. 6. Interrupt forwarding mechanism.

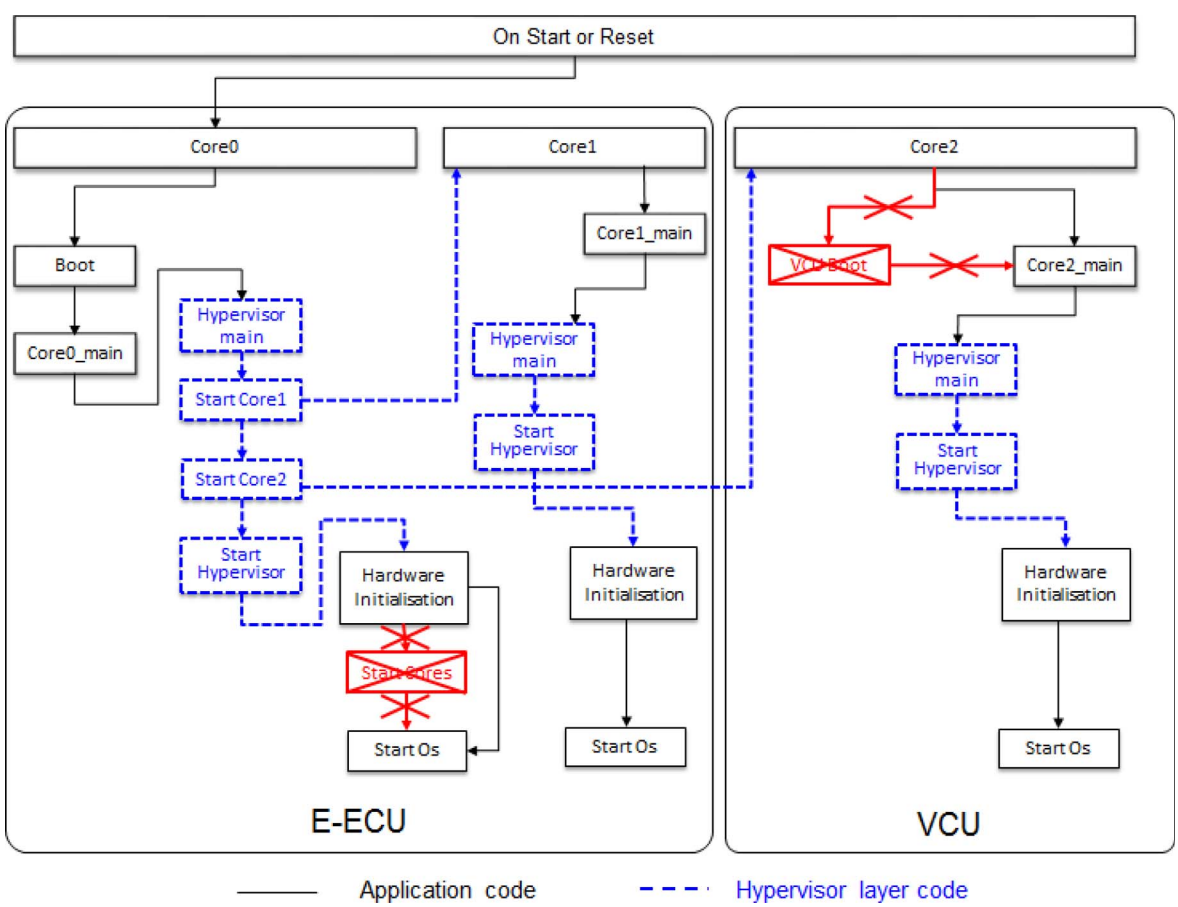

Fig. 5. Start-up sequence in a virtualized system.

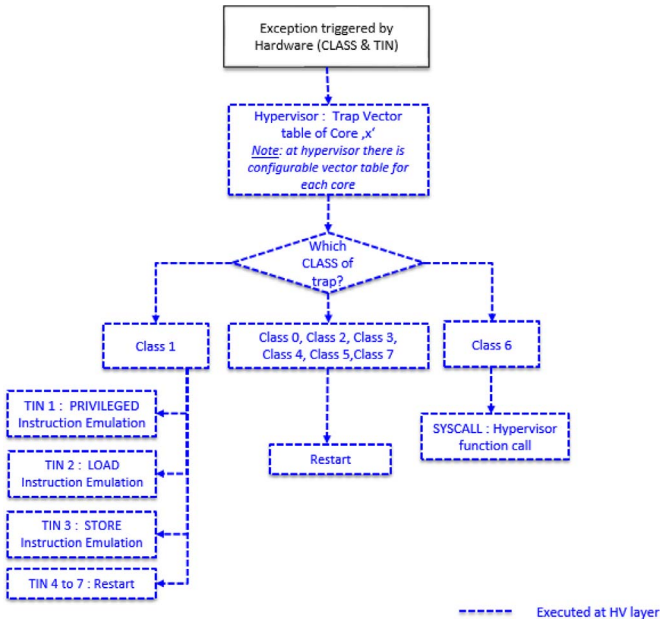

Fig. 7. Trap handling mechanism.

applications. It is possible to specify the actions for an exception (trap or interrupt) at the hypervisor vector table using the configuration files.

In case of an interrupt, there are two possible configurations at the hypervisor vector table

a) To handle the interrupt at hypervisor layer itself, or
b) To forward it for handling at the corresponding VM.

When a peripheral interrupts a core, the corresponding core execution switches to *supervisor* mode and jumps into hypervisor vector table. Based on the hypervisor vector table configuration, it executes the corresponding interrupt handler which is explained in Fig. 6. In this demonstrator, forwarding to the application happens via jump operation thus preventing secondary interrupts.

On the other hand, traps from the controller are exploited for emulating the input-output peripheral operation (IO emulation) and for handling privileged instructions. If an application tries to access a peripheral region, then controller will trigger a trap of class 1 for read or write violation. Similarly, a privileged instruction must be executed only in *supervisor* mode; otherwise a trap of class 1 will be triggered. (Fig. 7).

Just like interrupts, when a trap occurs–the corresponding core execution switches to *supervisor* mode and jumps into hypervisor trap vector table. Trap handlers are then executed in *supervisor* mode, which emulates the instruction and returns the result to the application from which the exception is triggered.

For example, in TC29x controller, instructions like move to core register (MTCR) and begin interrupt service routine (BISR) are some privileged instructions which are to be executed in *supervisor* mode. Since the applications are now running in *user-1* mode, MTCR and BISR instructions will result in a trap (Class 1 TIN 1). Trap is then routed to vector table of hypervisor, where the trap handler emulates the functions of MTCR and BISR. Common use cases of MTCR are used in access with program status word (PSW), interrupt control register (ICR), context information registers (PCX), etc.

4.5. Input/output (I/O) and peripheral access

Each application requires access to certain set of peripherals and input-output pins (termed as *ports* in TC29x). When the application was developed on an individual control unit, it had unrestricted access to the peripherals. However in a virtualized system, applications are running in *user-1* mode and the usage of peripherals has to be either dedicated or shared among the applications. Each peripheral is governed by a set of registers and these registers are mapped to a particular address range of the controller. In TC29x, peripheral registers are mapped to the address starting from 0xF000 0000 to 0xFFFF FFFF. Four different methods to facilitate the peripheral access are experimented i.e. MPU support, using hypervisor API, IO emulation method and virtual device emulator. Advantages and drawbacks of each method are briefly summarized at Table 3.

4.5.1. MPU support

Peripheral address ranges required dedicatedly for an application are configured under MPU through the hypervisor configuration files. Thereby the application gets direct access to this peripheral memory region in *user-1* mode itself. If another application tries to access this peripheral, hardware will trigger a trap. This method is limited by the number of MPU base/limit register pairs, as most of them are already used for partitioning flash and RAM regions (refer *Section 3.3* for usage details). *Note: This method is suitable for dedicated peripheral allocation.*

4.5.2. Hypervisor API or paravirtualization

Hypervisor provides APIs that has to be used in the application for accessing the peripheral memory regions. This is an intrusive technique, whereby the application has to be rewritten with the hypervisor API and recompiled. *For example, peripheral read operation to be replaced by Hv_Read and peripheral write operation to be replaced by Hv_Write API.*

Table 3
Comparing and contrasting the peripheral access methods.

Method	Benefits	Drawbacks
MPU Support	■ Applications have direct access to the peripheral memory in *user-1* mode itself, ■ Faster peripheral access time, ■ Non-intrusive technique.	○ Arbitration mechanism is not possible, ○ Suitable only for dedicated peripherals, ○ Demands higher MPU register sets from the controller.
Hypervisor API	■ Hypervisor APIs are open and developed according to the application.	○ This method works on a trap handler and it slows down the execution time, ○ Porting effort is high, as the entire driver code has to be modified (intrusive).
I/O emulation	■ Easy in porting the applications, as the underlying driver layers are not modified (non-intrusive), ■ Any number of IO address ranges can be configured and no limitations, ■ Supports sharing of peripherals.	○ Each time with change in instruction set architecture or inclusion of new load / store commands, the module for emulating the instruction has to be rewritten, ○ This method works on a trap handler and it slows down the execution time.
Virtual Device Emulator (VDE)	■ Arguments can be passed while calling the VDE API, ■ Arbitration and synchronization mechanism can be implemented.	○ Requires the entire driver implementation as part of VDE, ○ Requires modification in the software architecture, where the hypervisor should be made accessible to the application software (intrusive).

Application executes the hypervisor API, which in turn invoke a SYSCALL trap, switches from *user-1* to *supervisor* mode and performs the peripheral access through hypervisor function. Other side of this method is the complication in substituting every peripheral memory access instruction of the legacy applications.

4.5.3. IO emulation method

Substituting every peripheral memory access instruction with hypervisor API is complex and this IO emulation method brings a solution to it. This nonintrusive method focuses on emulating the trap handler to perform the peripheral access. Applications use *load* and *store* instructions to access the memory. Whenever application encounters a load or store instruction pointing to the memory region configured under IO, trap class 1 for read/write violation is triggered as described in Fig. 7. Then the controller switches from *user-1* mode to *supervisor* mode and jumps to the trap handler which is executed from hypervisor region. This trap handler emulates the load and store instruction. Advantages of this method are the support to shared resource usage and non-intrusive migration of the applications. It is also not limited by the number of address range specifiers. Drawback of this method is the need to modify emulation codes inside hypervisor with changes in instruction set architecture.

4.5.4. Virtual device emulator (VDE)

VDE is another user space inside hypervisor for real-time sharing of communication peripherals like CAN and Flexray. Arbitration and synchronizing mechanisms are implemented inside the VDE layer, facilitating direct access to peripheral memory region. Applications access a peripheral by calling VDE and VDE communicates back to the application using either: interrupts, shared memory or values returned in core registers.

4.5.5. MPU vs. IO method–an example

Performance of CAN communication over the bus depends on the type of access provided to the application. TC29x has a CAN controller which can support up to six CAN nodes, classified under two sub modules as listed in Table 4.

In a vehicular network, multiple control units communicate over a same CAN node. With six CAN nodes, six different communication networks can be realized. MPU and IO emulation are the two non-intrusive techniques and access to CAN nodes across E-ECU and VCU can be done as follows

MPU method:

a) Dedicated peripherals can only be configured under this method. So, either MultiCAN+ or MultiCANR has to be dedicatedly allocated to E-ECU and the other for VCU.
b) If the same CAN node is configured for E-ECU and VCU, then there will be data inconsistency. This implies, VCU and E-ECU cannot communicate over the same CAN node.
c) Application's CAN driver can directly access and modify the CAN registers in *user-1* mode itself.

IO emulation:

Table 4
MultiCAN modules in TC29x.

Module	Number of nodes	Number of message objects	Address range
MultiCAN+	4	256	0xF001 8000–0xF001 BFFF
MultiCANR+	2	128	0xF002 8000–0xF002 BFFF

a) Dedicated or shared peripherals can be configured under this method. So, it is possible for E-ECU and VCU to communicate over the same CAN node with data consistency.
b) When the CAN node is shared, then an additional arbitration logic (e.g. based on CAN msg id) has to be incorporated inside hypervisor layer.
c) Applications accessing the CAN registers will end up in trap and the trap handler performs the fetch/store operation.
d) Additional delay with respect to MPU method is expected with this implementation.
e) Another limitation is that function arguments cannot be passed over to arbitration logic and hence arbitration algorithm has to be implemented based on raw CAN data.

Typical use-case in a vehicular network is to share communication over the same CAN node and IO emulation method supports in this use-case. Experimental results discussed in next section perform a comparison between the two non-intrusive techniques, dedicated MPU and shared IO emulation of the peripherals.

5. Experimental results and analysis

To begin with, this section charts out the memory allocation against requirement of each application and hypervisor module. Next, this section elaborates on the performance of the virtualized system in terms of basic scheduling frame (BSF) CPU loading, overhead from hypervisor functions and runtime consumption of critical tasks and interrupts. Two types of input-output access, namely dedicated resource usage under MPU method and shared resource usage under IO emulation are assessed and compared. Also results between systems with and without hypervisor are taken into consideration at each evaluation point.

5.1. Memory allocation procedure

First step is to estimate the memory requirement of each application and the hypervisor module. Most of the industrial standard compilers have add-on tool sets, which run on the executable and linkable format file (ELF) and provide information on the labels and code size. Hypervisor code and labels are generated from the hypervisor library based on the configuration data. Hence, it is recommended to re-evaluate the hypervisor code and data size for every change in configuration. Hypervisor occupies only a smaller memory footprint, with major contribution of it from hypervisor configured interrupt and trap vector table. Table 5 summarizes the memory requirements of applications and hypervisor library with MPU and IO configuration used in this demonstrator. Additional buffer space should be accounted for user defined functions inside hypervisor, *for example: arbitration algorithm, VDE.*

Variables and stack are part of RAM sections, while constants, code and calibratable variables occupy the flash memory. Based on the evaluated memory requirements, RAM and flash regions are allocated

Table 5
Evaluated memory requirements.

	E-ECU software	VCU software	Hypervisor software with MPU support	Hypervisor software with IO emulation
Variables	332 KB	48 KB	12 KB*	13 KB*
Calibratable variables	466 KB	13 KB	–	–
Constants	442 KB	61 KB	–	–
Code size	3752 KB	439 KB	102 KB*	104 KB*
Stack requisite	45 KB	7 KB	not estimated	not estimated

* Hypervisor code size and labels differ based on the configuration.

Table 6
Memory allocation.

	Code flash	Data flash	Code RAM	Data RAM
E-ECU Software	5632 KB	1024 KB	80 KB	472 KB
VCU Software	832 KB	192 KB	16 KB	128 KB
Hypervisor	512 KB[+]	–	–	4 KB

[+] Additional space to accommodate user defined hypervisor functions.

Table 7
Overhead with hypervisor codes.

Functions	Execution time (μs)[+]
Function that checks for memory range	3.2
Emulation function for input-output access[*]	2.09–2.93
Emulation function for privilege instruction: MTCR	2.33
Emulation function for privilege instruction: BISR	1.36
Hypervisor main function	8.15

[+] Execution time is the maximum value observed over multiple iterations.
[*] Execution time for input-output emulation depends on the type of load and store instruction that is emulated. Here, maximum execution time of the fastest emulated instruction is 2.09μs and that of longest emulated instruction is 2.93μs.Hence, range of values is specified.

to the modules as presented in Table 6. An additional 28 KB of shared RAM is reserved for inter VM communication.

5.2. Overhead with hypervisor

As discussed in *Section 4.4*, hypervisor has its own vector tables which decide the handling of interrupts and traps. Further, hypervisor performs the emulation activities for load /store and privileged instructions. Hence it becomes essential to measure the overhead contributed by hypervisor functions. Based on TC29x ISA–13 types of *load*, 17 types of *store* and 4 types of privileged instructions are emulated in this demonstrator. With the help of hardware system timer (frequency 100 MHz), execution time consumed by the emulation functions are evaluated [39] and a summary is presented in Table 7.

5.3. Temporal consumption

Performance of a virtualized system in terms of core loading and timing parameters of tasks and interrupts are evaluated and compared. E-ECU behavior in a virtualized system is critical for our demonstration and the results with reference to E-ECU functions are presented here.

5.3.1. Influence on the CPU load

In multicore system, CPU load is a primary factor to determine the software distribution across cores and is defined as per Eq. (1).

$$CPU\ LOAD = [Time\ spent\ for\ executing\ code\ other\ than\ idle\ function\ /\ Observation\ period] \quad (1)$$

In this equation, observation period has a big influence on the meaning of CPU load. If the entire software execution is observed, then CPU load will converge to an average value. This measurement is beneficial but will miss the essential peak and drop points of the CPU load. Therefore the key to using CPU load is to define a right short-term period for observation and here the periodicity of tasks is exploited [40]. Our time frame for observation is a least common multiple (LCM) of all periodic tasks and this time frame is referred as basic scheduling frame (BSF). Therefore,

$$BSF\ CPU\ LOAD = [Time\ spent\ for\ executing\ code\ other\ than\ idle\ function/BSF\ period] \quad (2)$$

BSF CPU load is measured at three sample points (engine rpm = 0 rpm, 3000 rpm and 6500 rpm) in a closed loop [41] labcar environment. This experiment is performed for both non-virtualized and virtualized systems. Further, virtualized systems with dedicated IO access under MPU method and shared IO access under IO emulation are assessed for same sample points. The results are summarized graphically in Fig. 8.

To remind: during system consolidation, interrupt handlers and interrupt based segment synchronous tasks of E-ECU were allocated to *core 0* and time based safety critical tasks of E-ECU to *core 1*. (Fig. 3).

From Fig. 8, observable points are:

a) Virtualization with hardware MPU support still accounts for 8–10% increase in CPU load and this is mainly due to emulation of privileged instructions and involvement of additional vector tables at hypervisor layer,
b) At 6500 rpm, IO emulation technique stalls *core0* with a load of 100%. This high load is contributed by the emulation of load and store instructions and higher frequency of interrupt based segment synchronous tasks,
c) Steady increase in CPU load at *core 0* is due to frequent execution of interrupt based segment synchronous tasks with engine rpm.

5.3.2. Influence on hardware interrupts

Hardware interrupts are first processed at the hypervisor layer and then forwarded to the interrupt vector table of the corresponding application E-ECU or VCU. Performance to an interrupt is evaluated from its interrupt service routine (ISR). Five timing parameters are instrumented to evaluate the performance between non-virtualized and virtualized systems. Timing parameters for an ISR are (Fig. 9):

a) Gross execution time (GET) is the amount of time between the start and end of an ISR, including preemption,
b) Core execution time (CET) is the amount of processor time required to complete an ISR, independent of preemption,
c) Preemption time (PT) is the amount of time an activated task or interrupt is put on hold because of execution of another higher priority task or interrupts,
d) Latency time (LT) is the amount of time between the trigger of hardware interrupt to actual start of execution of the ISR,
e) Response time (RT) is the sum of gross execution time and latency time. In short, response time is the time from activation to completion of task or interrupt.

In this demonstrator, *core0* is handling the interrupts of E-ECU application and has a total of 91 interrupts configured to it. ISRs of ADC and CAN interrupts are instrumented and the results are presented in Tables 8 and 9 respectively. In virtualized system, ADC and CAN peripherals are set for dedicated access to E-ECU under MPU method and for shared access between E-ECU and VCU under IO emulation.

From the Tables 8 and 9, observable points are:

a) In the virtualized system, overhead of emulation functions contribute to the variation in timing parameters,
b) Virtualization with MPU support did not bring any improvement with respect to ISRs. There are multiple constraints an ISR goes through, like execution of ISR happens in *user-1* mode, emulation of privileged instructions and execution time at hypervisor vector table before being forwarded to the application,
c) Focusing on the latency time, virtualization has only brought longer waiting. This delay has to be compensated in the application design for the need of virtualization in hard real-time systems.

5.3.3. Influence on tasks

In automotive control system, typically two types of tasks are handled by the operating system: periodic tasks and segment synchronous

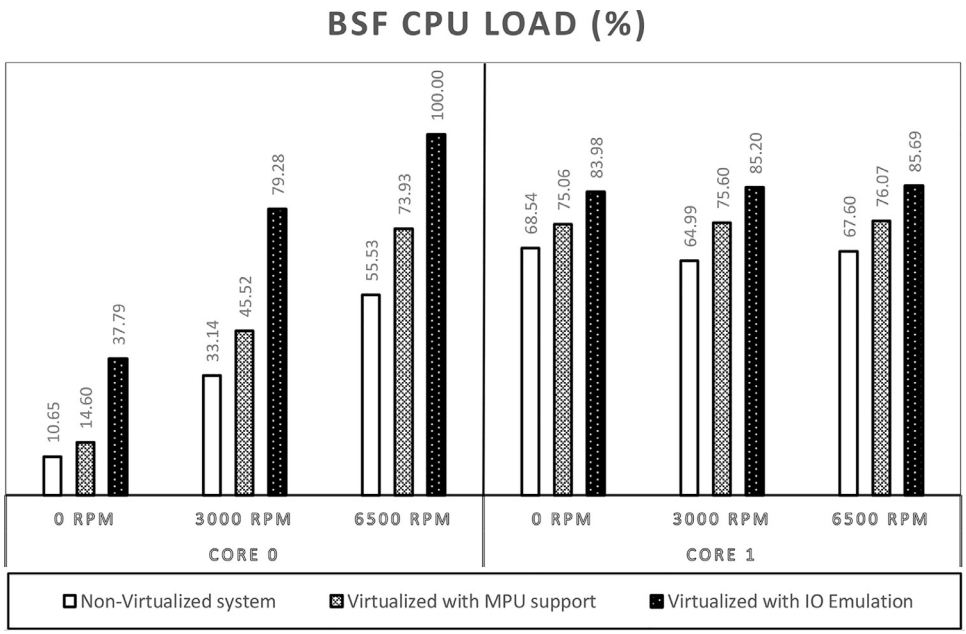

Fig. 8. Comparison of maximum BSF CPU load.

tasks. Periodic tasks are further classified into preemptive and co-operative tasks, activating the tasks at their statically configured timeslots. Segment synchronous task on the other hand, works based on the engine position and are communicating with operating system using interrupts. Only on reception of interrupt, these tasks are activated. At higher engine speed, frequency of the segment synchronous tasks activation also increases.

Execution time of the tasks is measured in a closed loop [41] labcar environment, subjected to a full load scenario: engine speed: 6500 rpm and vehicle cruising at 220kmph. The results are presented in Table 10.

From the Table 10, observable points are:

a) Execution time for virtualization with IO emulation is on higher side, suggesting the necessity for additional support from the hardware,
b) In contrary to the ISR behavior seen in *Section 5.3.2*, virtualization with MPU support brings better results in case of handling time period tasks. This is primarily due to direct hardware access in *user-1* mode provided by the MPU configuration.

6. Summary

Innovations at automotive industry trend towards a centralized computation power. This will be conceivable by consolidating functions into smaller units called *domains*, then consolidating these smaller domains to a bigger domain and so on, until the centralized unit is achieved. Typical consolidation will involve integrating the functional components into single software and porting onto the hardware. Standards like AUTOSAR encourage this integration by normalizing the interfaces. However, integrating systems will end up in complex software, lacking isolation from the application's interference.

Table 8
Timing parameters for ADC interrupt (average value).

Configurations	GET (μs)	CET (μs)	PT (μs)	LT (μs)	RT (μs)
Non-virtualized system	10.79	10.34	7.33	0.45	10.79
Virtualized with MPU support	69.70	34.12	103.04	35.58	69.70
Virtualized with IO Emulation	99.71	45.90	128.61	53.81	99.71

Table 9
Timing parameters for CAN interrupt (average value).

Configurations	GET (μs)	CET (μs)	PT (μs)	LT (μs)	RT (μs)
Non-Virtualized system	8.93	8.77	0.00	0.16	8.93
Virtualized with MPU support	1043.68	249.45	470.93	794.24	1043.68
Virtualized with IO Emulation	1224.19	332.62	592.11	891.58	1224.19

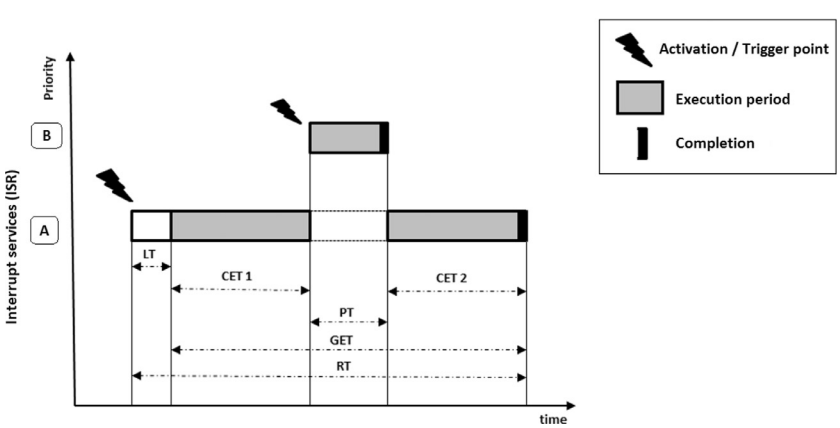

Fig. 9. Diagrammatic illustration of the timing parameters with respect to ISR 'A' ; GET–gross execution time, CET = CET 1 + CET 2 core execution time, PT–preemption time, LT–Latency time, RT–response time.

Table 10
Average execution time of tasks.

Tasks	Non-virtualized system (μs)	Virtualized with MPU support (μs)	Virtualized with IO emulation (μs)
Periodic task 1 ms (preemptive)	83.34	90.45	127.50
Periodic task 10 ms (preemptive)	4530.89	5221.59	5651.25
Periodic task 100 ms (cooperative)	2646.33	2821.41	2920.03
Segment synchronous task 1	924.24	991.25	1086.55
Segment synchronous task 2	928.07	910.34	954.68

6.1. Role of hypervisor

Automotive control system involves software components of different ASIL category, where isolation from interference becomes the heart of the system. In this demonstrator, two legacy control systems of automotive domain are consolidated onto a single hardware without merging their functional units and each system operates independently. Hypervisor works like a firewall between the applications and between the hardware and application. Any interactions have to pass through this hypervisor layer.

Implementing virtualization in an automotive state-of-the-art multicore controller without ISA for virtualization is a challenge. Usage of MPU register sets from hardware and controller's operating mode (*user* and *supervisor*) has proven us in realizing the isolation. Another form of the isolation, where a restart due to trap or reset on VCU does not restart the E-ECU is also achieved in our hypervisor demonstration.

6.2. Recap on the implementation

Each core is treated as a virtual machine and has its own AUTOSAR configurable memory regions, interrupt and trap vector tables and peripherals to use. MPU register pairs are used in isolating the flash and RAM regions between the applications and hypervisor. In case of access violation, a trap is triggered by the hardware. Traps are first handled in hypervisor vector table. Trap handlers are also used as emulators for performing load and store operation in peripheral region. Privilege instructions are also handled in similar way by emulating the trap handler.

Peripherals communicate to the core via interrupts. Interrupt handling mechanism in this demonstrator shows the possibility of handling in hypervisor layer or forwarded to the application level. Hardware also facilitates the core selection for an interrupt. Another essential part is the modification in boot up sequence (Fig. 5), where hypervisor starts earlier and knows the existence of its interacting cores.

6.3. Shortcomings and recommendations

In this demonstration, we came across several challenges and some of the critical ones are listed in this subsection.

Cost for hypervisor implementation:

a) Porting the system onto the desired hardware, which involves changes in the BSW layer (software near hardware and in terms of AUTOSAR, this software layer is referred as MCAL),
b) Unification and adaptation in the application for hypervisor [11], where direct memory access has to be replaced with hypervisor APIs,
c) Configuration of each VM under hypervisor, in terms of memory layout, interrupt and traps handlers and access to peripheral devices,
d) Additional performance cost, evident from measures of BCF CPU load, hypervisor function overheads and runtime cost to interrupts and tasks.

Boundary constraints:

a) Applications will execute only in lesser privileged *user-1* mode and will not have access to peripheral region. Only the hypervisor functions execute in *supervisor* mode with all privileges,
b) Even in case of interrupt to be handled at the application, it still has to go via hypervisor vector table and then forwarded. This increases the interrupt response time,
c) Shared resources have to go through a contention protocol and this slows down the access,
d) Spatial protection works based on MPU configuration. Limitation in the MPU register sets prevents from micro assignment of hardware peripherals,
e) Scheduling interrupts and segment synchronous tasks onto the same core, causes an overload situation at higher engine rpm,
f) MCAL layer is still within each application and static memory addressing is used by hypervisor to access the MCAL functions,
g) Privileged instructions when used by applications have to be emulated via hypervisor trap handlers.

Recommendations:

a) Spreading the interrupt services across cores will reduce the BSF CPU load,
b) Targeting the interrupts directly to the core containing ISR will reduce the response time,
c) Constructing a centralized MCAL layer for the hardware and integrating under hypervisor, will improve the overall execution time and avoid static addressing based hypervisor access,
d) Utilizing a hardware which has ISA for virtualization, separate controller modes and memory management unit (MMU) will improve the response time of the system.

7. Conclusion

We have demonstrated a hypervisor that runs on an automotive state-of-the-art multicore controller, which is not designed for virtualization use cases. This controller is idiomatic for the automotive domain with safety level ASIL D. So the ability to run a hypervisor on such hardware is a critical evaluation. In this demonstrator, hypervisor handles three VMs in parallel, with spatial and temporal separation and achieving the functionality of both the applications. We have evaluated the virtualization cost in terms of, consumption of BSF CPU load, overhead from hypervisor functions and runtime cost to interrupts and tasks with hypervisor techniques. Although certain key operations were longer than usual, as a system the applications met their deadlines and provided a good behavior in our tests.

Virtualization approach is always a potential solution for consolidating disparate systems onto the same hardware and keeping them isolated from interference. However, costs of porting the legacy systems and time delays from virtualization have to be taken into consideration. We are currently devising an experiment on real-time sharing of CAN controller between E-ECU and VCU using virtual device emulator (VDE) [42,43], which is an extension to hypervisor for user based arbitration and synchronization function implementation.

References

[1] R. Dominik, D. Kaule, M. Kucera, Achieving a scalable e/e-architecture using autosar and virtualization, SAE Int. J. Passenger Cars-Electron. Electric. Syst. 6 (2013) 489–497.
[2] G. Weiss, P. Schleiss, C. Drabek, Towards flexible and dependable E/E-architectures for future vehicles, International Workshop on Critical Automotive Applications: Robustness & Safety (CARS 2016), 2016.

[3] R. Hamann, S. Kriso, K. Williams, J. Klarmann, J. Sauler, ISO 26262 Release Just Ahead: Remaining Problems and Proposals for Solutions, SAE Technical Paper, SAE International, 2011, , http://dx.doi.org/10.4271/2011-01-1000.
[4] D. Reinhardt, M. Kucera, Domain Controlled architecture-a new approach for large scale software integrated automotive systems, PECCS, 2013.
[5] AUTOSAR - Classic Platform. [Online]. http://www.autosar.org/standards/classic-platform/.
[6] G. Heiser, Hypervisors for consumer electronics, Consumer Communications and Networking Conference, IEEE, 2009.
[7] H. Sutter, The free lunch is over: a fundamental turn toward concurrency in software, Dr. Dobb's J. 30 (2005) 202–210.
[8] A.K. Sundar Rajan, S.K. Vasudevan, NirmalaDevi M., Trends in task allocation for multicore system, Int. J. Electric. Comput. Eng. 6 (2016) 3018.
[9] M. Strobl, M. Kucera, A. Foeldi, T. Waas, N. Balbierer, C. Hilbert, Towards automotive virtualization, Applied Electronics (AE), IEEE, 2013.
[10] G. Heiser, The role of virtualization in embedded systems, Proceedings of the 1st Workshop on Isolation and Integration in Embedded Systems, ACM, 2008.
[11] D. Reinhardt, G. Morgan, An embedded hypervisor for safety-relevant automotive E/E-systems, Industrial Embedded Systems (SIES), IEEE, 2014.
[12] AURIX™ Family – TC29xT. [Online]. http://www.infineon.com/cms/en/product/microcontroller/32-bit-tricore-tm-microcontroller/aurix-tm-family/aurix-tm-family-%E2%80%93-tc29xt/channel.html?channel=db3a304342c787030142dc92c9aa1674.
[13] A. Aguiar, F. Hessel, Embedded systems' virtualization: the next challenge? Rapid System prototyping (RSP), IEEE, 2010.
[14] A. Patel, M. Daftedar, M. Shalan, M. El-Kharashi, Embedded hypervisor xvisor: a comparative analysis, Parallel, Distributed and Network-Based Processing (PDP), IEEE, 2015.
[15] M. Mounika, C. Chinnaswamy, A Comprehensive review on embedded hypervisors, Int. J. Adv. Res. Comput. Eng. Technol. (IJARCET) 5 (5) (2016).
[16] A. Masrur, S. Drossler, T. Pfeuffer, S. Chakraborty, VM-based real-time services for automotive control applications, Embedded and Real-Time Computing Systems and Applications (RTCSA), IEEE, 2010.
[17] A. Crespo, et al., Multicore partitioned systems based on hypervisor, IFAC Proceedings, Elsevier, 2014.
[18] Miguel Masmano, Ismael Ripoll, Alfons Crespo, and J. Metge, "Xtratum: a hypervisor for safety critical embedded systems," 2009.
[19] G. Heiser, B. Leslie, The OKL4 microvisor: convergence point of microkernels and hypervisors, Proceedings of the First ACM Asia-Pacific Workshop on Workshop on Systems, ACM, 2010.
[20] Christoffer Dall and Jason Nieh, "KVM/ARM: experiences building the linux ARM hypervisor," 2013.
[21] K. Gilles, S. Groesbrink, D. Baldin, T. Kerstan, Proteus hypervisor: full virtualization and paravirtualization for multi-core embedded systems, International Embedded Systems Symposium, Springer, 2013.
[22] Integrity Multivisor. [Online]. http://www.ghs.com/products/rtos/integrity_virtualization.html.
[23] (2008, April) The Freescale Embedded Hypervisor ("Topaz"). [Online]. http://wiki.jackslab.org/images/Topaz_overview.pdf.
[24] (2011) Wind River Hypervisor. [Online]. https://www.windriver.com/products/product-notes/PN_Hypervisor_0611.pdf.
[25] G. Neiger, A. Santoni, F. Leung, D. Rodgers, R. Uhlig, Intel virtualization technology: hardware support for efficient processor virtualization, Intel Technol. J. 10 (2006).
[26] P. Varanasi, G. Heiser, Hardware-supported virtualization on ARM, Proceedings of the Second Asia-Pacific Workshop on Systems, ACM, 2011.
[27] Roberto Mijat and Andy Nightingale, "Virtualization is coming to a platform near you," 2011.
[28] XEN development portal. [Online]. https://wiki.xen.org/wiki/Xen_Project_Software_Overview.
[29] semiengineering group. [Online]. http://semiengineering.com/why-you-need-asil-certified-processor-ip-for-automotive-safety-applications.
[30] Freescale. [Online]. https://www.nxp.com/docs/en/data-sheet/MPC5643L.pdf.
[31] STM microelectronics. [Online]. http://www.st.com/en/automotive-microcontrollers/spc5-32-bit-automotive-mcus.html?querycriteria=productId=SC963.
[32] Texas Instrument. [Online]. http://www.ti.com/lsds/ti/microcontrollers-16-bit-32-bit/c2000-performance/safety/tms570/overview.page.
[33] Aurix - Infineon. [Online]. https://www.infineon.com/cms/en/product/microcontroller/32-bit-tricore-tm-microcontroller/aurix-tm-family/aurix-tm-family-%E2%80%93-tc29xt/channel.html?channel=db3a304342c787030142dc92c9aa1674.
[34] Renesas. [Online]. https://www.renesas.com/en-in/products/microcontrollers-microprocessors/rh850/rh850p1x/rh850p1hc.html.
[35] ARM developer site. [Online]. https://developer.arm.com/products/processors/cortex-r/cortex-r52.
[36] ARM market survey. [Online]. https://www.arm.com/files/pdf/ARM_Addresses_1B_Powertrain_Processor_Market.pdf.
[37] J. Fisher-Ogden, Hardware Support For Efficient Virtualization, San Diego, 2006.
[38] C. Moratelli, F. Hessel, S. Filho, Hardware-assisted interrupt delivery optimization for virtualized embedded platforms, Electronics, Circuits, and Systems (ICECS), IEEE, 2015.
[39] M. Aichouch, J. Prevotet, F. Nouvel, Evaluation of the overheads and latencies of a virtualized RTOS, Industrial Embedded Systems (SIES), IEEE, 2013.
[40] Peter Gliwa, "Timing analysis and timing verification today and in the future," 2014.
[41] ETAS HIL. [Online]. https://www.etas.com/en/products/labcar_system_components.php.
[42] C. Herber, D. Reinhardt, A. Richter, A. Herkersdorf, HW/SW trade-offs in I/O virtualization for Controller Area Network, Design Automation Conference (DAC), 2015.
[43] A. Kohn, et al., Timing analysis for hypervisor-based I/O virtualization in safety-related automotive systems, SAE Int. J. Passenger Cars-Electron. Electric. Syst. 10 (2017).

Arun Kumar Sundar Rajan is an architect in Robert Bosch Engineering and Business Solutions, India. He received B.Tech degree in Electronics and Communication engineering from Amrita Vishwa Vidyapeetham and later M.E Degree in Embedded System Technology from Anna University Chennai–Regional Office, Coimbatore. He has nine years of experience in developing software for automotive embedded systems at Bosch. He is also a Ph.D. scholar in electronics and communication engineering department at Amrita Vishwa Vidyapeetham University. His research interest includes real-time systems, operating systems and automotive control system.

Armin Feucht is a software developer at Bosch Engineering GmbH, Germany. He has received diploma degree in Software Engineering at University Heilbronn, Germany. He has nine years of experience in automotive embedded systems. He worked on many projects for different customers in many topics around basis software, like operating system, hardware configuration and software sharing.

Lothar Gamer is a team leader in Robert Bosch GmbH, Germany. He has twenty five years of experience in automotive embedded systems. He worked on many projects, handling software sharing architectures like AUTOSAR for different customers. His expertise are in embedded software architecture, real-time embedded system and software sharing models.

Smaili Idriz received the B.S. and M.S. degree in computer science and telecommunication from the University of Prishtina, Republic of Kosova, in 1995, and the Ph.D. degree in computer science from the Vienna University of Technology, Vienna, Austria, in 2004. From 2004 to 2008 he was a professional software engineer at CTBTO (UN organization) in Vienna. He joined Robert BOSCH Group as a software and system engineer in 2008. Since 2013 he is a software project leader leading software projects for the BMW Group. His research interests include (embedded) software engineering, operating systems, fault-tolerance and real-time systems.

M. Nirmala Devi is a professor in the department of Electronics and Communication Engineering at Amrita Vishwa Vidyapeetham, Coimbatore, India. Her research interest include VLSI Design and Testing, Computational Intelligence, Hardware Security and Trust, Evolvable Hardware and RF CMOS System Design. She has published around 55 papers in the International Journals and Conferences in her field of expertise. She has served as the reviewer for refereed international conferences and international journals which include the following; Springer Journal of the Institution of Engineers (India): Series B, Inderscience Int. Journal of Information and Communication Technology. She is the recipient of the following awards–Marquis Who's Who in the World–2011 and 2000 Outstanding Intellectuals of the 21st Century–2011–International Biographical Center, Cambridge, England. She has received the financial grant for the research proposal from Defense Research & Development Organization, Delhi, India.